IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts Science Commerce and Management

High Level Design

on

Identity Security Framework for Agriculture

Student Name: Aparna A.N

Industry Mentor: Mr. Shashank **Guided by: Mr. Vikyath Shetty**

Table of Contents

1. Introduction. 3
   1.1. Scope of the document. 3
   1.2. Intended Audience 3
   1.3. System overview. 3

2. System Design. 4
   2.1. Application Design 4
   2.2. Process Flow. 4
   2.3. Information Flow. 5
   2.4. Components Design 6
   2.5. Key Design Considerations 6
   2.6. API Catalogue. 6

3. Data Design. 7
   3.1. Data Model 7
   3.2. Data Access Mechanism 7
      3.3. Data Retention Policies 7
   3.4. Data Migration 7

4. Interfaces 7

5. State and Session Management 8

6. Caching 8

7. Non-Functional Requirements 8
   7.1. Security Aspects 8
   7.2. Performance Aspects 9

8. References 9

1. Introduction

FarmConnect is a security-first digital ecosystem designed to modernize and secure agricultural transactions. This framework provides a centralized platform for agricultural sales and user management, featuring a robust Role-Based Access Control (RBAC) system. It distinguishes between Farmers, who can securely trade crops and receive real-time notifications, and Administrators, who hold elevated control to audit transactions and manage user account lifecycles

1.1 Scope of the Document

This document defines the high-level architecture, security protocols, and operational workflows of the FarmConnect platform. It serves as the primary blueprint for understanding how the application manages agricultural commerce while maintaining a robust identity security framework.

1.2 Intended Audience

- System Architects: To review the integration of security and data persistence.
- Lead Developers: To guide the implementation of modular logic.
- Project Managers: To understand the functional capabilities and user flows.

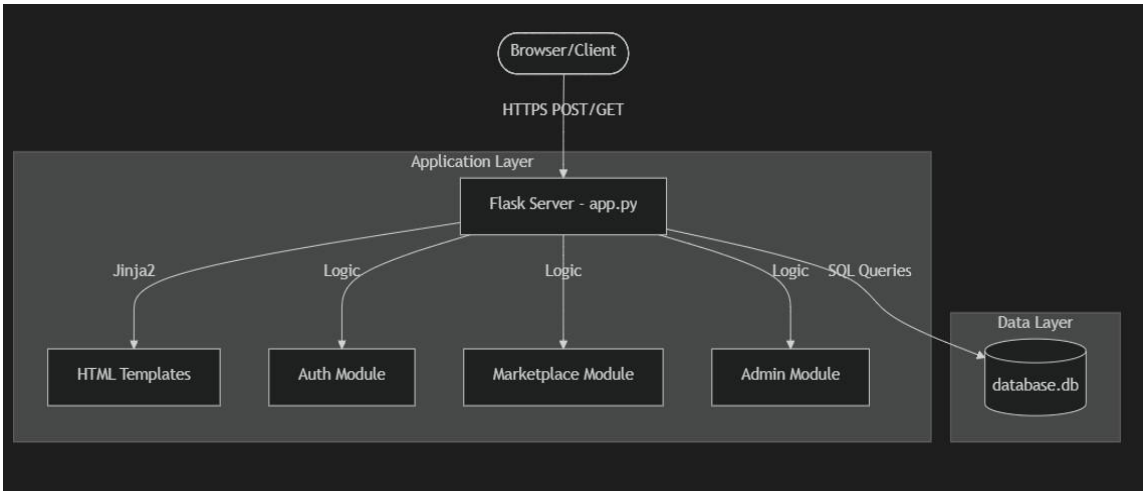
1.3 System Overview

FarmConnect is a dedicated digital marketplace designed to bridge the gap between farmers and administrative oversight. The system operates on a Role-Based Access Control (RBAC) model:

- Farmers: Can register, browse a curated crop catalog, place orders, and receive real-time status updates via a notification system.
- Administrators: Hold elevated privileges to manage the user lifecycle (including deactivating accounts), audit transactions, and approve or reject orders.

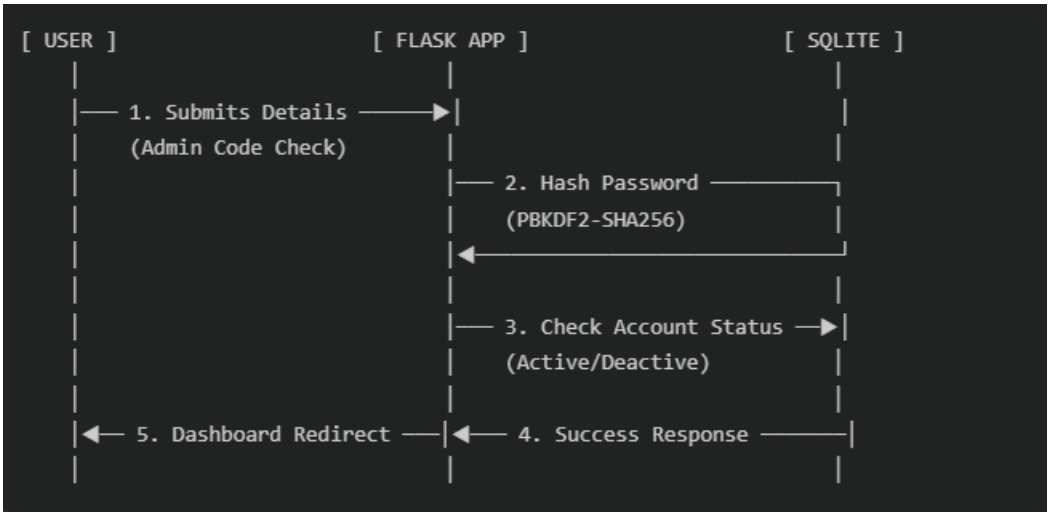
2. System Design

2.1 Application Design

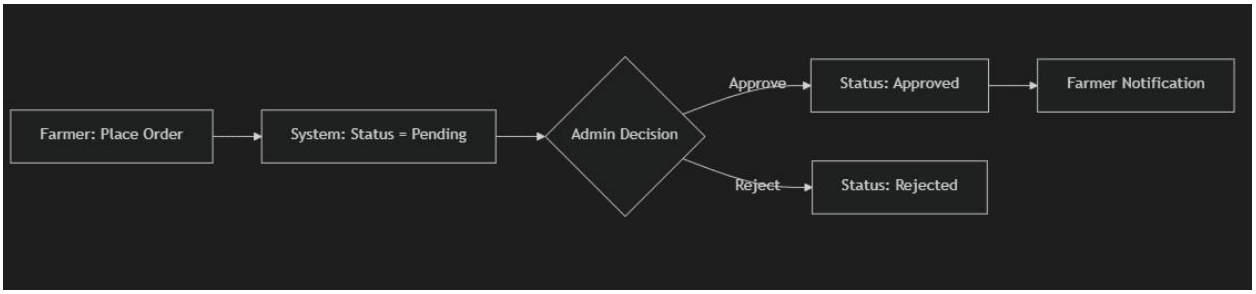


2.2 Process Flows

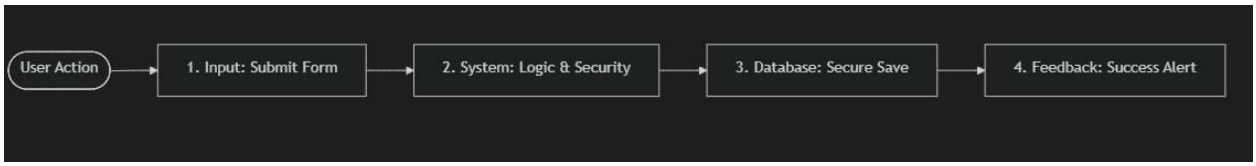
A. Secure Authentication (Signup/Login)



B . Order Lifecycle (Farmer to Admin)



2.3 Information Flow



Phase 1: User Input

- The user fills out a form on the website (like placing a crop order or signing up).
- This data travels securely from the browser to our Flask server.

Phase 2: Security & Logic (Behind the Scenes)

- The system checks the data for errors (like verifying email formats.
- For passwords, the system "locks" them using high-level encryption so they are never stored in plain text.

Phase 3: Data Saving

- The system uses a direct bridge (SQL) to write this information into the database.db file. This ensures your information is saved permanently even if the server restarts.

Phase 4: Real-Time Feedback

- The next page the user sees will show a bright, clear alert (e.g., "Order Placed!") so the user knows exactly what happened.

2.4 Component Design

- Auth Module: Handleshashing, Flask-Login session management, and role validation.
- Marketplace Module: Manages the hardcoded crop catalog and dynamic order placement logic.
- Admin Panel: Provides the interface for is_active_account toggling, user deletion, and order auditing.

2.5 Key Design Considerations

- Security First: All user-provided data is treated as untrusted until validated and hashed.
- Direct Control: Uses raw SQL instead of an ORM for maximum transparency and performance.
- Zero Latency: Local SQLite file ensures data retrieval is nearly instantaneous.

2.6 API Catalogue

Feature	Description	Access Level
Identity Management	Secure Signup, Login, and Logout functionality.	Public / Authenticated
Marketplace	Browsing the crop catalog and placing purchase orders.	Farmer Only
User Oversight	Global view of all users and account activity status.	Admin Only
Feature	Description	Access Level

Sales Auditing	**Detailed view of all transactions for approval or rejection.**	**Admin Only**

3. Data Design

3.1 Data Model

User Entity: Stores personal details, roles, and the is_active_account status (used for the Kill-Switch).
Order Entity: Stores transaction logs including crop name, quantity, price, and the current lifecycle status (Pending/Approved/Rejected).
Notification Entity: A new entity that stores messages for users, linked to their User ID, with an is_read flag for tracking.

3.2 Data Access Mechanism

FarmConnect utilizes the native Python sqlite3 driver with a Row Factory configuration. This ensures that the application can access records using column names (e.g., row['email']), making the code more maintainable compared to index-based lookups.

3.3 Data Retention Policies

The system maintains a permanent log of all transactions for auditing purposes. Records are only purged if an administrator explicitly performs a "Delete" action, which also triggers a cascading cleanup of that user's orders and notifications..

3.4 Data Migration

The system uses the fix_db.py script to handle initialization and schema updates. This ensures that the transition from a basic structure to a production-ready ecosystem is handled without data corruption.

4. Interfaces

- Frontend Dashboard
- Emerald Green Theme: Representing the agricultural focus.

- Responsive Grid: Mobile-first design using Bootstrap
- Visual Feedback: Real-time flash messages for every system action (e.g., "Order Placed Successfully")

➢ Database Interface

- Direct filesystem I/O between the Flask runtime and the database.db file, ensuring zero network latency for data retrieval.

5. State and session Management

Sessions are managed via Flask-Login using Signed Secure Cookies. This ensures that session data cannot be tampered with on the client side. The server-side logic also tracks last_login to provide admins with insights into user engagement.

6. Caching

- Client-Side Caching: Static assets like the Emerald Green CSS and images are served with headers that allow browsers to cache them, reducing load times.
- Kernel Caching: The system relies on the internal SQLite and OS file buffers to ensure that frequently accessed data (like the crop list) is retrieved at maximum speed.

7. Non-Functional Requirements

7.1 Security Aspects

- SQL Safety: 100% of database interactions use Parameterized Queries to prevent SQL Injection.
- Account Locking: Administrators can instantly lock or unlock any account without deleting the user's data.
- Input Validation: Strict Regex patterns are used to validate email formats before they reach the database layer.

7.2 Performance

- Edge Computing Logic: By using a local SQLite instance, the system avoids the latency associated with network-based databases, providing instantaneous response times for local farm management.
- Optimized Rendering: The use of Jinja2 templates ensures that pages are rendered server-side and delivered to the browser with minimal processing overhead.

8. References

- **Flask Documentation**
- **Python sqlite3 Library**